

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет информационных технологий и управления

Кафедра систем управления

Дисциплина: Основы машинного обучения

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему:

**«Прогнозирование загруженности тренажерного зала на основе
моделей регрессии»**

Выполнил:

ст. гр. 461402 Радюш А. А.

Проверил:

Калоша Н. И.

Минск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ОПИСАНИЕ ДАТАСЕТА И АРХИТЕКТУРЫ ПРОЕКТА	4
1.1 Описание датасета	4
1.2 Архитектура и программная реализация проекта	5
2 МАТЕМАТИЧЕСКОЕ ОПИСАНИЕ МОДЕЛЕЙ.....	10
2.1 Линейная регрессия	10
2.2 Ядерная регрессия.....	14
2.3 Случайный лес (композиция решающих деревьев и бэггинг)	18
3 СРАВНИТЕЛЬНЫЙ АНАЛИЗ МОДЕЛЕЙ	23
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	27

ВВЕДЕНИЕ

Описание проекта и инструкция к запуску так же продублированы в файле README.md

Описание проекта: учебный проект посвящен разработке и исследованию моделей машинного обучения для предсказания точного количества людей в тренажерном зале в определенный момент времени на основе дня недели, времени суток, температуры на улице и начала семестра. Данный проект, мог бы лечь в основу реального приложения для фитнес-клуба, показывающего оптимальное время для тренировок без очередей.

Цели проекта:

- сравнить эффективность различных методов регрессии;
- выявить наиболее точную модель для данного датасета;
- визуализировать предсказания моделей для понимания их работы.

Используемые модели. Для решения задачи были реализованы и обучены следующие методы:

- линейная регрессия (OLS) – классический аналитический метод;
- линейная регрессия (SGD) – итеративный метод градиентного спуска;
- ядерная регрессия (RBF) – метод, позволяющий улавливать нелинейные зависимости с помощью радиально-базисного ядра;
- случайный лес (Random Forest) – ансамблевый метод, разбивающий данные на логические интервалы (кусочно-постоянная аппроксимация).

Для запуска проекта на Unix-подобной системе или WSL (Windows SubSystem for Linux) выполните следующие команды:

1. Для установки необходимых библиотек выполните следующее: перейдите в директорию проекта и выполните команды: (для установки на Debian/Ubuntu системах, на других используйте свой пакетный менеджер)

```
sudo apt install python3.12-venv -y
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2. После чего запустить main.py:

```
python main.py
```

1 ОПИСАНИЕ ДАТАСЕТА И АРХИТЕКТУРЫ ПРОЕКТА

1.1 Описание датасета

<https://www.kaggle.com/datasets/nsrose7224/crowdedness-at-the-campus-gym>

Выгруженный файл .csv датасета располагается в data/data.csv

Для решения поставленной задачи прогнозирования и проведения сравнительного анализа моделей машинного обучения был использован открытый набор данных «Crowdedness at the Campus Gym» (Загруженность университетского тренажерного зала), собранный в одном из университетских кампусов США и опубликованный на платформе Kaggle.

Объектом исследования в данном датасете является посещаемость фитнес-центра, зафиксированная с интервалом в несколько минут на протяжении двух лет (период с августа 2015 года по март 2017 года). Исходный объем выборки составляет 62184 наблюдений, что обеспечивает высокую статистическую значимость обучающих и тестовых множеств.

Структура и метрики данных.

Каждый объект выборки (строка) содержит информацию о конкретном моменте времени и сопутствующих факторах. Вектор признаков включает в себя как непрерывные числовые, так и категориальные (бинарные) переменные.

Целевая переменная (Target Variable):

- number_people (целочисленное значение) – количество посетителей, находящихся в тренажерном зале в момент фиксации данных.

Входные признаки (Предикторы):

- date (временная метка) – точная дата и время фиксации наблюдения;

- timestamp (числовой признак) – количество секунд, прошедших с начала суток (используется для точного позиционирования внутри дня);

- hour (целочисленный признак) – час суток (от 0 до 23), отражающий суточную цикличность;

- day_of_week (категориальный признак) – день недели (от 0 до 6, где 0 – понедельник), отражающий недельную цикличность;

- is_weekend (бинарный признак) – флаг выходного дня (1 – суббота/воскресенье, 0 – будний день);

- is_holiday (бинарный признак) – флаг официального праздничного дня (1 – праздник, 0 – рабочий день);

- temperature (числовой признак) – температура наружного воздуха (в градусах Фаренгейта), отражающая погодные условия;

- is_start_of_semester (бинарный признак) – флаг первых недель академического семестра (период максимальной активности студентов);

- is_during_semester (бинарный признак) – флаг протекания учебного семестра (включение/исключение каникулярного времени);

month (целочисленный признак) – месяц года (от 1 до 12), отражающий годовую сезонность.

Особенности и специфика данных.

Набор данных обладает рядом специфических особенностей, усложняющих задачу математического моделирования:

Высокая степень нелинейности зависимостей: Посещаемость тренажерного зала не связана с фактором времени линейно. Распределение имеет выраженный мультимодальный (многопиковый) характер – утренние и вечерние часы пик сменяются глубокими спадами в ночное время и умеренным затишьем в обеденные часы.

Взаимосвязь разнородных факторов: На целевую переменную одновременно влияют циклы разного порядка (суточные, недельные, семестровые) в сочетании со случайными внешними факторами (например, колебания температуры воздуха).

Наличие шума: Поскольку фиксация данных происходила непрерывно, в выборке присутствуют естественные флуктуации (шум), вызванные случайными задержками посетителей, кратковременными изменениями расписания и т.д.

Преимущества датасета для целей исследования.

Выбор датасета «Crowdedness at the Campus Gym» для выполнения данного курсового проекта обоснован следующими архитектурными и методологическими преимуществами:

Идеальный полигон для сравнительного анализа разнородных алгоритмов: Из-за сильной нелинейности и выраженных пиков загруженности данный датасет позволяет наглядно продемонстрировать ограничения классических подходов (таких как OLS и SGD регрессии) и раскрыть потенциал более сложных методов – ядерной регрессии (Kernel Ridge) и ансамблей решающих деревьев (Random Forest).

Сбалансированный объем выборки: Наличие десятков тысяч записей позволяет полноценно обучать ресурсоемкие модели. При этом объем данных позволяет эффективно применять технику стратификации и выделения подвыборок (сэмплинга) для оптимизации тяжелых гиперпараметров алгоритмов методом кросс-валидации (GridSearchCV).

Практическая применимость и интерпретируемость результатов: Задача прогнозирования человеческих потоков является классической бизнес-задачей. Результаты моделирования имеют понятную физическую интерпретацию, что позволяет качественно оценить бизнес-метрики моделей (например, среднюю абсолютную).

1.2 Архитектура и программная реализация проекта

1.2.1 Программный комплекс разработан на языке программирования Python 3.12 с использованием принципов модульной архитектуры и разделения обязанностей (Separation of Concerns). Основная идея такой архитектуры заключается в изолировании отдельных этапов решения задачи (загрузка

данных, конфигурация моделей, визуализация) в независимые программные модули, управление которыми делегировано центральному управляющему скрипту.

Обоснование и преимущества модульной структуры:

Высокая читаемость и сопровождаемость кода (Maintainability): Каждый модуль отвечает строго за один логический этап пайплайна (Single Responsibility Principle). Изменение логики построения графиков или добавление новых признаков в предобработку не затрагивает код обучения моделей.

Масштабируемость (Scalability): Архитектура позволяет легко расширять список исследуемых алгоритмов регрессии, добавляя новые функции-конструкторы в изолированный модуль моделей без модификации логики проведения экспериментов.

Дерево директории проекта имеет следующую структуру:

```
my_gym_course_project/
├── data/
│   └── data.csv          # Исходный набор данных
├── src/                  # Пакет с исходным кодом модулей
│   ├── __init__.py      # Маркер инициализации Python-пакета
│   ├── data_loader.py   # Модуль загрузки и обработки данных
│   ├── models.py        # Модуль инициализации моделей
│   └── visualization.py # Модуль генерации графики
├── main.py              # Главный управляющий скрипт
├── requirements.txt      # Перечень библиотек
└── README.md            # Документация и инструкция по запуску
```

Ниже приведено детальное описание каждого компонента системы, его внутреннего состава и роли в общем жизненном цикле программы.

1.2.2 Модуль `src/data_loader.py` (Слой работы с данными).

Отвечает за физическое чтение файла данных, первичную очистку и подготовку матриц признаков для обучения.

Функция `get_prepared_data` считывает файл `data/data.csv`, выделяет целевой вектор `number_people`, удаляет неинформативные текстовые поля (например, исходную строку даты), формирует матрицу признаков, осуществляет разделение выборки на обучающее и тестовое множества в заданном соотношении (например, 80/20) с фиксацией генератора случайных чисел для воспроизводимости. Возвращает кортеж из четырех массивов: `X_train`, `X_test`, `y_train`, `y_test`.

```
def get_prepared_data(file_path='data/data.csv'):
    df = pd.read_csv(file_path)

    # Отделяем целевую переменную (Y) от признаков (X)
```

```

y = df['number_people']

# Удаляем целевую переменную из матрицы признаков,
# а также удаляем столбец 'date', так как это текст, а
линейная регрессия ест только числа.
X = df.drop(columns=['number_people', 'date'])

# Прячем 20% данных для финальной проверки
return train_test_split(X, y, test_size=0.2,
random_state=42)

```

1.2.3 Модуль src/models.py.

В этом файле сосредоточены функции, конструирующие объекты моделей с предустановленными или оптимально подобранными параметрами.

Основные функции:

`get_linear_regression()` – инициализирует и возвращает объект классической линейной регрессии (OLS).

`get_sgd_regression()` – инициализирует и возвращает объект линейной регрессии, обучаемой методом стохастического градиентного спуска.

`get_kernel_regression(X_sample, y_sample)` – принимает репрезентативную подвыборку данных, выполняет процедуру поиска оптимальных гиперпараметров (коэффициента регуляризации, параметров RBF-ядра) методом кросс-валидации и возвращает настроенный объект ядерной регрессии.

`get_random_forest(X_sample, y_sample)` – принимает подвыборку данных, запускает сеточный поиск (`GridSearchCV`) для оптимизации структуры ансамбля (глубина деревьев, критерии расщепления) и возвращает сконфигурированный объект Случайного леса для финального обучения.

1.2.4 Модуль src/visualization.py.

`plot_model_behavior_over_time(models_dict, X_train, y_train)` – функция построения графиков. Принимает словарь обученных моделей, исходную обучающую матрицу и целевой вектор. Функция генерирует синтетический профиль (идеальный день), фиксируя средние значения побочных факторов и непрерывно изменяя час суток от 0 до 23. На основе предсказаний каждой модели на этом профиле строятся четыре координатных графика поверх облака реальных точек данных. Результат экспортируется в графический файл высокого разрешения `models_visualization.png`.

1.2.5 Модуль main.py.

Является точкой входа в программу и выполняет роль оркестратора. Он не содержит в себе непосредственной логики вычислений, а координирует вызовы функций из пакета `src`.

Выполнение программы происходит по последовательному сценарию, управляемому через `main.py`:

Пользователь инициирует выполнение скрипта `main.py`.

`main.py` обращается к `data_loader.py` и получает подготовленные массивы обучающих и тестовых данных.

```
X_train, X_test, y_train, y_test = get_prepared_data()
```

В `main.py` выделяется небольшая случайная подвыборка (`X_sample`), которая передается в функции модуля `models.py`.

```
# Берутся 5000 строк из тренировочного набора данных
X_sample = X_train.iloc[:5000]
y_sample = y_train.iloc[:5000]
```

Модуль `models.py` выполняет оптимизацию параметров и возвращает сконфигурированные объекты моделей обратно в `main.py`.

```
# Словарь моделей для сравнения
models_to_test = {
    "Linear Regression (Analytic)": get_linear_regression(),
    "Linear Regression (SGD)": get_sgd_regression(),
    "Kernel Regression (RBF)":
get_kernel_regression(X_sample, y_sample),
    "Random Forest": get_random_forest(X_sample, y_sample)
}
```

`main.py` последовательно отправляет каждую модель в функцию `run_experiment()`, где происходит их обучение на полном объеме данных (`X_train`), тестирование (`X_test`), фиксация времени работы и метрик ошибок.

```
def run_experiment(model_name, model_object, X_train, X_test,
y_train, y_test):
    # обучаем модель, высчитывая время
    start_time = time.time()
    model_object.fit(X_train, y_train)
    training_time = time.time() - start_time

    # Оценка
    predictions = model_object.predict(X_test)
    mae = mean_absolute_error(y_test, predictions)
    r2 = r2_score(y_test, predictions)

    return {"Время (сек)": training_time, "MAE": mae, "R2": r2}

results = {}
for name, model in models_to_test.items():
```



```
        results[name] = run_experiment(name, model, X_train,
X_test, y_train, y_test)
```

Результаты расчетов выводятся на экран в виде сравнительной таблицы.

```
res_df = pd.DataFrame(results).T
    print("\nФИНАЛЬНОЕ СРАВНЕНИЕ МОДЕЛЕЙ:")
    print(res_df)
```

На финальном этапе `main.py` передает все обученные модели и обучающую выборку в модуль `visualization.py`, который генерирует итоговый график и сохраняет его на диск, завершая жизненный цикл программы.

```
# Отрисовка графиков
    # Модели в словаре models_to_test уже обучены в цикле выше
    plot_model_behavior_over_time(models_to_test, X_train,
y_train)
```

2 МАТЕМАТИЧЕСКОЕ ОПИСАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ МОДЕЛЕЙ

2.1 Линейная регрессия (аналитический метод МНК и стохастический градиентный спуск)

Линейная регрессия – это классический метод восстановления непрерывной зависимости, предполагающий, что целевая переменная y может быть аппроксимирована линейной комбинацией входных признаков x с некоторыми весовыми коэффициентами w .

Пусть X – матрица «объект-признак» размерности $l \times n$, где l – количество объектов в выборке, а n – количество признаков. Вектор истинных ответов (в рамках данного проекта – количество людей в тренажерном зале) обозначим как $y \in \mathbb{R}^1$. Задача алгоритма заключается в поиске вектора весов $w \in \mathbb{R}^n$, минимизирующего отклонение предсказаний модели $\hat{y} = Xw$ от реальных значений.

2.1.1 Метод наименьших квадратов (МНК).

Поскольку решить систему линейных уравнений $Xw = y$ абсолютно точно в реальных задачах невозможно из-за наличия шума и избыточности данных, ставится задача минимизации вектора ошибок (остатков) $(Xw - y)$. Для измерения длины вектора ошибок в линейной алгебре традиционно применяется евклидова норма. Таким образом, глобальная цель оптимизации записывается следующим образом:

$$\|Xw - y\|^2 \rightarrow \min_w$$

В покомпонентной записи для всей выборки это выражение преобразуется в функционал качества – среднеквадратичную ошибку (Mean Squared Error, MSE), которая и выступает в качестве функции потерь $Q(w)$:

$$Q(w) = (1/l) \sum_{i=1}^l (\langle w, x_i \rangle - y_i)^2$$

где x_i – вектор признаков i -го объекта выборки,
 y_i – его истинное целевое значение.

Аналитическое решение (Нормальное уравнение).

Для нахождения оптимального вектора весов w^* необходимо взять производную от матричного выражения функции потерь $\|Xw - y\|^2$ по вектору весов w и приравнять её к нулю (необходимое условие экстремума функции). В результате этого матричного дифференцирования выводится точное аналитическое решение, называемое нормальным уравнением:

$$w^* = (X^T X)^{-1} X^T y$$

Несмотря на математическую строгость и изящность, прямое применение аналитического решения на практике сталкивается с двумя фундаментальными проблемами:

Вычислительная сложность: Нахождение обратной матрицы $(X^T X)^{-1}$ требует проведения порядка $O(n^3)$ арифметических операций. При обработке огромных датасетов с тысячами признаков вычислительные затраты становятся критическими, приводя к переполнению оперативной памяти.

Мультиколлинеарность: Если в исходных данных присутствуют признаки, сильно зависящие друг от друга (например, температура, выраженная одновременно в градусах Цельсия и Фаренгейта), определитель матрицы $X^T X$ становится близок к нулю. Матрица становится вырожденной (плохо обусловленной), что делает процедуру ее численного обращения невозможной или крайне нестабильной.

В Python (библиотека NumPy) эта формула записывается буквально слово в слово:

```
# np.linalg.inv(...) - функция находит обратную матрицу
w = np.linalg.inv(X.T @ X) @ X.T @ y
```

Хотя матричный подход отлично объясняет суть, в реальных задачах и в проекте никто не пишет формулу умножения матриц вручную.

Программная реализация линейных моделей вынесена в изолированный модуль `src/models.py`. Для построения пайплайна использовалась библиотека промышленного анализа данных `scikit-learn`. Была написана функция, возвращающая объект модели:

```
from sklearn.linear_model import LinearRegression

def get_linear_regression():
    print("\nОбучаем модель Линейной Регрессии...")
    # модель под капотом будет использовать матричную формулу
    w^* = (X^T X)^{-1} * X^T * y
    return LinearRegression(fit_intercept=True)
```

В функции параметр `fit_intercept=True` указывает алгоритму на необходимость автоматического расчета свободного коэффициента (сдвига/bias), что геометрически позволяет прямой линии регрессии не проходить строго через начало координат (0,0), подстраиваясь под среднее значение целевой переменной.

2.1.2. Стохастический градиентный спуск (SGD)

Для преодоления ограничений аналитического подхода используется итерационный метод оптимизации – градиентный спуск. Вместо мгновенного аналитического вычисления глобального минимума алгоритм начинает движение со случайного вектора весов w и на каждом шаге корректирует его в

сторону, противоположную вектору градиента функции потерь (направление наискорейшего убывания ошибки):

$$w \leftarrow w - \eta \nabla Q(w)$$

где $\eta > 0$ – темп (шаг) обучения (learning rate), регулирующий скорость сходимости,
 $\nabla Q(w)$ – вектор градиента функции потерь.

В модификации стохастического градиентного спуска (Stochastic Gradient Descent, SGD) градиент на каждом шаге оптимизации вычисляется не по всей огромной выборке, а по одному случайно выбранному объекту (или мини-батчу), что радикально ускоряет вычисления и позволяет модели обучаться на непрерывных потоках данных. Шаги повторяются до тех пор, пока алгоритм не достигнет сходимости (когда изменения весов или функции потерь становятся пренебрежимо малыми).

Вот как этот цикл выглядит в коде (на чистом Python / NumPy)

```
# ИНИЦИАЛИЗАЦИЯ
# Начинаем со случайных весов (пусть пока оба равны нулю)
# w[0] - сдвиг (базовое количество), w[1] - влияние часа
w = np.array([0.0, 0.0])

learning_rate = 0.001 # Размер шага (скорость спуска)
epochs = 100          # Сколько раз мы будем обновлять веса

# ЦИКЛ ОБУЧЕНИЯ (Спуск по воронке)
for epoch in range(epochs):

    # Выбираем случайную строку из данных (Стохастический
    # подход)
    random_index = np.random.randint(len(X))
    x_i = X[random_index]
    y_i = y[random_index]

    # Шаг А: Предсказание текущей моделью (Скалярное
    # произведение)
    # prediction = w0*1 + w1*час
    prediction = np.dot(x_i, w)

    # Шаг Б: Вычисление ошибки
    error = prediction - y_i

    # Шаг В: Вычисление градиента
    # Математически градиент MSE для одного объекта равен: 2 *
    # ошибка * значение_признака
    gradient = 2 * error * x_i

    # Шаг Г: Обновление весов
```

```
# Делаем шаг в сторону, противоположную градиенту (вниз)
w = w - learning_rate * gradient
```

В реальных задачах писать циклы вручную не принято. Для этого в библиотеке `scikit-learn` есть готовый класс `SGDRegressor`. Была написана функция, возвращающая объект модели с жестко зафиксированным параметром `random_state` для обеспечения полной воспроизводимости результатов на любой вычислительной системе.

Здесь появляется одно критически важное правило: градиентный спуск ненавидит данные разного масштаба. Если один признак измеряется в единицах (часы: 10, 18), а другой в тысячах (зарплата: 50000, 120000), «воронка ошибок» становится вытянутой, и алгоритм начинает метаться из стороны в сторону, не находя дна.

Поэтому перед применением SGD данные всегда нужно масштабировать (приводить к одному диапазону).

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDRegressor

def get_sgd_regression():
    print("\nОбучаем модель через Стохастический Градиентный
    Спуск (SGD)...")
    # make_pipeline объединяет масштабирование и саму модель в
    один понятный объект
    # Теперь данные внутри будут автоматически нормализоваться
    перед каждым шагом спуска
    return make_pipeline(StandardScaler(), SGDRegressor(
        loss='squared_error',
        max_iter=1000,
        tol=1e-3,
        random_state=42)
    )
```

В функции `get_sgd_regression` в качестве аргумента оптимизации `loss` передается значение `'squared_error'`, что полностью соответствует минимизации функционала квадратичной ошибки, описанного математически в формуле $Q(w)$. Параметр `max_iter=1000` задает жесткий лимит на количество эпох (полных проходов по выборке) для предотвращения заикливания, а критерий остановки `tol=1e-3` останавливает итерационный процесс раньше, если функция потерь перестает уменьшаться на заданную величину, экономя процессорное время.

Оба сконфигурированных объекта обладают унифицированным программным интерфейсом `.fit()` и `.predict()`, что позволяет главному оркестратору `main.py` бесшовно запускать их оценку в едином цикле экспериментов.

2.2 Ядерная регрессия (непараметрическое сглаживание и приближение Нистрёма)

2.2.1 Математический аппарат непараметрической ядерной регрессии.

Главным ограничением классической линейной регрессии, описанной в предыдущем разделе, является необходимость жесткого априорного задания параметрической структуры модели (линейной функции). Если истинная зависимость в данных имеет сложный, мультимодальный характер (как в случае суточных пиков посещаемости тренажерного зала), линейная модель демонстрирует неудовлетворительное качество аппроксимации (высокое смещение).

Для преодоления этой проблемы применяется непараметрическая регрессия, в основе которой лежит концепция локального сглаживания. Вместо поиска единой глобальной аналитической формулы для всего пространства признаков, непараметрический алгоритм аппроксимирует целевую зависимость локально, вычисляя значение прогноза как взвешенное среднее ответов наиболее близких исторических точек.

Непараметрическая оценка Надарая–Ватсона

Для математической формализации этого принципа используется классическая формула ядерного сглаживания Надарая–Ватсона. Для произвольного запроса x прогноз модели $a_h(x; X^1)$ вычисляется по следующему правилу:

$$a_h(x; X^1) = (\sum_{i=1}^1 y_i K(\rho(x, x_i)/h)) / (\sum_{i=1}^1 K(\rho(x, x_i)/h))$$

где x – вектор текущего запроса (точка, в которой требуется составить прогноз).

x_i – вектор признаков i -го объекта из обучающей выборки X^1 .

y_i – истинный ответ (загруженность зала) для исторического объекта x_i .

$\rho(x, x_i)$ – метрика расстояния в признаковом пространстве между точкой запроса и обучающим примером.

$K(\tau)$ – функция ядра (весовая функция), определяющая закон убывания влияния исторической точки по мере увеличения расстояния до неё.

$h > 0$ – ширина окна сглаживания (масштабный коэффициент/bandwidth).

Суть формулы заключается в нормировании весов: сумма всех весов в знаменателе приводит масштаб результирующего прогноза к исходным единицам измерения целевой переменной.

Функция ядра $K(\tau)$.

Функция ядра определяет геометрическую форму распределения весов объектов. В зависимости от выбора ядра меняется гладкость результирующей кривой регрессии:

Прямоугольное ядро: $K(r) = \frac{1}{2}[|r| \leq 1]$. Присваивает всем точкам внутри окна h равные веса, из-за чего функция прогноза становится кусочно-постоянной (ступенчатой).

Квадратичное ядро (Епанечникова): $K(r) = \frac{3}{4}(1 - r^2)[|r| \leq 1]$. Плавно снижает вес объектов до нуля при приближении к границе окна.

Гауссовское ядро (RBF): $K(r) = \frac{1}{\sqrt{2\pi}} \exp(-r^2/2)$. Обладает бесконечным носителем (учитывает все точки выборки, но экспоненциально уменьшает их вес с ростом расстояния), обеспечивая максимальную гладкость и непрерывность функции прогноза.

В рамках данного проекта используется Гауссовское (Radial Basis Function, RBF) ядро.

Влияние ширины окна h и метод скользящего контроля (LOO).

Ширина окна сглаживания h является ключевым гиперпараметром, регулирующим баланс между смещением и дисперсией модели:

Слишком малое значение h : Окно становится чрезмерно узким. Алгоритм учитывает только изолированные ближайшие точки, фактически соединяя их напрямую. Модель подстраивается под случайные флуктуации и шум выборки, что ведет к переобучению (overfitting). Ошибка на обучающей выборке стремится к нулю, но на тестовых данных модель демонстрирует полную неадекватность.

Слишком большое значение h : Окно охватывает избыточное количество объектов. Алгоритм начинает глобально усреднять разнородные данные (например, смешивать утренние спады и вечерние пики посещаемости), превращая кривую прогноза в прямую горизонтальную линию. Это приводит к недообучению (underfitting).

Для поиска оптимального параметра h нельзя оценивать ошибку на той же выборке, на которой модель «обучается» (так как при $h \rightarrow 0$ ошибка будет нулевой). Для честной оценки применяется критерий выпуклой валидации Leave-One-Out (LOO). Суть алгоритма заключается в циклическом исключении каждого i -го объекта из выборки объемом l , настройке модели по оставшимся $l - 1$ объектам и расчете ошибки на исключенном объекте:

$$Q(h) = \sum_{i=1}^l (y_i - \hat{y}_{-i}(x_i, h))^2 \rightarrow \min_h$$

где $\hat{y}_{-i}(x_i, h)$ – прогноз в точке x_i , вычисленный без учета самого i -го объекта.

Оптимальным признается значение h , минимизирующее функционал $Q(h)$.

Проблема вычислительной сложности и приближение Нистрёма.

Прямое применение классического ядерного метода (например, алгоритма KernelRidge) к исходному датасету сталкивается с вычислительным барьером. Классический алгоритм требует вычисления матрицы размерности $l \times l$, где l – число объектов обучающей выборки.

Для исследуемого датасета после разбиения объем обучающей выборки составляет $l = 49\,747$ строк. Попытка построить матрицу приводит к созданию двумерного массива размера: $9\,747 \times 49\,747 = 2\,474\,765\,709$ ячеек

Выделение такого объема непрерывной памяти под одну операцию вызывает критическую ошибку переполнения (`Unable to allocate 18.4 GiB for an array...`) на стандартных вычислительных системах.

Для преодоления данного ограничения в проекте применено приближение Нистрёма (Nystroem Approximation). Это эффективный метод снижения размерности ядерных матриц. Суть метода заключается в случайном выборе небольшого подмножества из M опорных объектов ($M \ll l$, в проекте $M = 1000$) – так называемых «анкеров». Вместо полной матрицы $l \times l$ вычисляется матрица расстояний между всеми объектами и опорными точками (размерность всего $l \times M$), а затем исходное пространство признаков линейно проецируется в это низкоразмерное ядерное подпространство. На результирующих признаках обучается эффективная линейная модель с регуляризацией (Ridge), что позволяет полностью сохранить математические свойства RBF-гладкости, снизив затраты ОЗУ.

2.2.2 Программная реализация ядерной регрессии.

Напишем ядро Надарая-Ватсона на чистом NumPy.

```
# Функция ядра (Гауссово)
def gaussian_kernel(distance, h):
    return np.exp(-(distance**2) / (2 * h**2))

# Предсказание (Реализация формулы Надарая-Ватсона)
def predict_kernel_regression(x_new, X_train, y_train, h):
    # Считаем расстояния от новой точки до всех обучающих
    distances = np.abs(x_new - X_train)
    # Считаем веса по ядру
    weights = gaussian_kernel(distances, h)
    # Формула взвешенного среднего
    return np.sum(weights * y_train) / np.sum(weights)

y_pred = [predict_kernel_regression(x, X, y, h) for x in X_test]
```

Программная реализация непараметрического алгоритма интегрирована в изолированный пакет проекта и представлена функцией `get_kernel_regression(X_sample, y_sample)`, расположенной в модуле `src/models.py`.

Полный исходный код функции имеет следующий вид:

```
from sklearn.preprocessing import StandardScaler
from sklearn.kernel_ridge import KernelRidge
from sklearn.model_selection import GridSearchCV
```



```

from sklearn.kernel_approximation import Nystroem
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline

def get_kernel_regression(X_sample, y_sample):
    print("\nНачинаем обучение модели через Ядерную Регрессию.
    Это может занять некоторое время...")

    scaler_sample = StandardScaler()
    X_sample_scaled = scaler_sample.fit_transform(X_sample)

    param_grid = {
        'gamma': [0.4, 0.55, 0.7, 0.85, 1], # Различная ширина
окна
        'alpha': [0.155, 0.17, 0.185, 0.2] # Сила регуляризации
(штраф за сложность)
    }

    print("\tЗапуск GridSearchCV на подвыборке 5000 строк...")
    grid_search = GridSearchCV(
        KernelRidge(kernel='rbf'),
        param_grid,
        cv=3,
        scoring='neg_mean_squared_error',
        n_jobs=-1
    )
    grid_search.fit(X_sample_scaled, y_sample)

    best_gamma = grid_search.best_params_['gamma']
    best_alpha = grid_search.best_params_['alpha']

    print("\t--- Лучшие параметры найдены ---")
    print(f"\tОптимальная gamma: {best_gamma}")
    print(f"\tОптимальная alpha: {best_alpha}\n")

    return make_pipeline(
        StandardScaler(),
        Nystroem(kernel='rbf', gamma=best_gamma,
n_components=500, random_state=42),
        Ridge(alpha=best_alpha)
    )

```

Логику работы представленного программного кода можно разделить на четыре взаимосвязанных этапа:

Стандартизация признаков подвыборки (Feature Scaling):

Применение класса `StandardScaler()` выполняет центрирование и нормирование данных, приводя каждый признак к нулевому математическому ожиданию и единичной дисперсии. На данном этапе это происходит локально для матрицы `X_sample` с помощью метода `.fit_transform()`.

Оптимизация гиперпараметров методом сеточного поиска (Grid Search):

Для поиска оптимального баланса между смещением и дисперсией (предотвращения переобучения и недообучения) формируется сетка параметров `param_grid`. Поиск ведется по двум направлениям:

Коэффициент `gamma` определяет ширину окна сглаживания. Исследуется диапазон от 0.4 до 1.0.

Коэффициент `alpha` отвечает за регуляризацию, исследуется диапазон от 0.155 до 0.2.

Сеточный поиск инициализируется через объект `GridSearchCV` над базовой моделью `KernelRidge(kernel='rbf')`. Стратегия кросс-валидации фиксируется на уровне `cv=3` (трехблочный кросс-контроль. Критерием оптимизации выступает минимизация среднеквадратичной ошибки `scoring='neg_mean_squared_error'`. Использование параметра `n_jobs=-1` дает команду операционной системе производить параллельные вычисления на всех доступных ядрах процессора. Процедура запускается методом `.fit()` на репрезентативной подвыборке в 5000 строк, что исключает долгий простой вычислительной системы.

Экстракция наилучших конфигураций:

После завершения кросс-валидации из атрибута `.best_params_` извлекаются скалярные значения `best_gamma` и `best_alpha`, обеспечившие минимальную ошибку на валидационных блоках. Их численные значения выводятся в консоль для обеспечения прозрачности и логирования хода эксперимента.

Композиция финального вычислительного конвейера (Pipeline):

Функция возвращает не просто обученную модель, а готовый пустой конвейер `make_pipeline()`, настроенный на оптимальные гиперпараметры. Это архитектурное решение инкапсулирует в себе всю цепочку трансформации данных, которая будет автоматически применена в `main.py` при вызове метода `.fit()` уже на полном объеме обучающей выборки (`X_train`):

Первый слой (`StandardScaler()`) — масштабирует всю входящую матрицу `X_train`.

Второй слой (`Nystroem(...)`) — осуществляет нелинейное проецирование данных в ядерное подпространство с использованием найденного значения `gamma` и ограничением в `n_components=500` случайных центроидов (опорных объектов). Параметр `random_state=42` обеспечивает жесткую воспроизводимость проекции.

Третий слой (`Ridge(alpha=best_alpha)`) — оценивает линейные коэффициенты регрессии методом наименьших квадратов с регуляризацией в новом 500-мерном пространстве признаков.

2.3 Случайный лес (композиция решающих деревьев и бэггинг)

2.3.1 Математический аппарат ансамблирования и случайного леса.

Главная проблема непараметрической ядерной регрессии, описанной в предыдущем разделе, заключается в экстремально высокой чувствительности

к размерности пространства признаков и вычислительной перегрузке при расчете попарных расстояний. В качестве альтернативного нелинейного подхода, устойчивого к шуму и масштабу признаков, выступает Случайный лес (Random Forest) — ансамблевый метод машинного обучения, основанный на объединении ответов множества базовых решающих деревьев.

Математическая модель регрессионного решающего дерева.

Базовым алгоритмом в ансамбле является бинарное решающее дерево $b(x)$. Процесс его построения заключается в рекурсивном разбиении признакового пространства на непересекающиеся подмножества, называемые областями R_m .

Таким образом, математически решающее дерево реализует кусочно-постоянную аппроксимацию целевой зависимости.

Концепция бэггинга (Bootstrap Aggregating)

Одиночные решающие деревья обладают важным свойством: при глубоком разбиении они имеют крайне низкое математическое смещение (bias), но очень высокую дисперсию (variance) — они легко переобучаются под малейшие шумы в данных. Чтобы снизить дисперсию без увеличения смещения, применяется метод бэггинга.

Из исходной обучающей выборки объема l случайным образом с возвращением извлекается N бутстрэп-выборок (размера l каждая). Из теории вероятностей следует, что в каждую такую выборку попадает примерно 63.2% уникальных объектов исходного датасета. На каждой бутстрэп-выборке независимо обучается свое решающее дерево $b_n(x)$. Финальный прогноз ансамбля $A(x)$ для задачи регрессии формируется путем простого усреднения ответов всех построенных деревьев:

$$A(x) = (1/N) \sum_{n=1}^N b_n(x)$$

Согласно закону больших чисел, усреднение большого количества независимых случайных величин с одинаковым математическим ожиданием радикально снижает общую дисперсию модели.

Метод случайных подпространств (Случайный лес)

Если обучать деревья просто на бутстрэп-выборках, они получатся сильно скоррелированными (похожими друг на друга), так как алгоритм построения дерева будет раз за разом выбирать одни и те же наиболее сильные предикторы (например, час суток или температуру). Корреляция между базовыми алгоритмами препятствует снижению дисперсии ансамбля.

Для декорреляции деревьев Лео Брейман объединил бэггинг с методом случайных подпространств. Суть модификации заключается в том, что при построении каждого сплита (разбиения вершины) внутри каждого дерева выбор оптимального признака j и порога t ищется не среди всего множества n признаков, а среди случайного подмножества размера k

Этот случайный выбор признаков гарантирует разнообразие внутренней структуры деревьев, обеспечивая высокую обобщающую способность ансамбля при анализе сложных, нелинейных зависимостей.

2.3.2 Программная реализация модели случайного леса

Программная реализация ансамблевого алгоритма вынесена в функцию `get_random_forest(X_sample, y_sample)` модуля `src/models.py`. Конструирование модели и оптимизация ее структурных ограничений реализованы с помощью классов `RandomForestRegressor` и `GridSearchCV` из библиотеки `scikit-learn`.

Исходный код программного модуля имеет следующий вид:

```
from sklearn.ensemble import RandomForestRegressor from
sklearn.model_selection import GridSearchCV

def get_random_forest(X_sample, y_sample):
    print("\nНачинаем обучение модели через Случайный лес...")

    # max_depth ограничивает рост дерева (от переобучения)
    # min_samples_split - сколько минимум объектов должно быть в
    листе, чтобы его поделить
    param_grid = {
        'n_estimators': [50, 100, 150],    # Количество базовых
        деревьев в лесу
        'max_depth': [10, 15, 20, None],
        'min_samples_split': [2, 5, 10]
    }

    base_model = RandomForestRegressor(random_state=42, n_jobs=-
1)

    print("\tЗапуск GridSearchCV на подвыборке")
    grid_search = GridSearchCV(
        estimator=base_model,
        param_grid=param_grid,
        cv=3,
        scoring='neg_mean_absolute_error',
        n_jobs=-1
    )

    grid_search.fit(X_sample, y_sample)

    best_n_estimators = grid_search.best_params_['n_estimators']
    best_depth = grid_search.best_params_['max_depth']
    best_split = grid_search.best_params_['min_samples_split']

    print("\t--- Лучшие параметры найдены ---")
    print(f"\tОптимальное кол-во деревьев (n_estimators):
{best_n_estimators}")
    print(f"\tМаксимальная глубина (max_depth): {best_depth}")
```

```

print(f"\tМин. объектов для разделения (min_samples_split):
{best_split}\n")

return RandomForestRegressor(
    n_estimators=best_n_estimators,
    max_depth=best_depth,
    min_samples_split=best_split,
    random_state=42,
    n_jobs=-1
)

```

Математико-программный анализ этапов работы функции:

Инвариантность к масштабу данных (Отсутствие `StandardScaler`):

Важнейшей архитектурной особенностью данной функции является полное отсутствие предварительного масштабирования признаков. В отличие от линейной (МНК, SGD) и ядерной регрессий, операции в узлах решающего дерева представляют собой обычные пороговые предикаты. Математический результат логического сравнения абсолютно инвариантен к любым монотонным преобразованиям признаков. Это избавляет пайплайн от необходимости использования `StandardScaler` и исключает вычислительные погрешности округления.

Конфигурация пространства поиска (Параметры ансамбля):

Настройка ансамбля осуществляется через тонкую балансировку трех параметров в словаре `param_grid`:

`n_estimators`: определяет число N независимых деревьев. Рост этого параметра не ведет к переобучению случайного леса (что является его главным преимуществом), но монотонно снижает дисперсию, асимптотически приближая ошибку к некоторому минимуму. Выбран диапазон [50, 100, 150].

`max_depth`: ограничивает глубину деревьев для явного контроля объема использования оперативной памяти и защиты от чрезмерной детализации шумов. Значение `None` позволяет деревьям расти до максимального истощения выборки.

`min_samples_split`: математический регуляризатор, запрещающий разделение вершины, если в ней осталось меньше заданного числа объектов выборки. Позволяет эффективно бороться с переобучением на «листьях» дерева.

Оркестрация параллельного поиска:

Для проведения кросс-валидации инициализируется объект `GridSearchCV`. В качестве базового оценщика передается `RandomForestRegressor`. Параметр `random_state=42` гарантирует жесткую фиксацию генератора случайных чисел при формировании бутстрап-выборок и подпространств признаков, обеспечивая идентичность результатов при перезапусках. Аргумент `n_jobs=-1` прописан как на уровне самого регрессора, так и на уровне сеточного поиска. Это обеспечивает двухуровневое распараллеливание: ядра процессора одновременно вычисляют разные комбинации параметров сетки, а внутри каждой комбинации параллельно

строятся независимые деревья леса, что обеспечивает максимальную утилизацию ресурсов многопроцессорных систем.

По завершении итерационного перебора функция извлекает оптимальные параметры, логирует их структуру в консоль и возвращает полностью сконфигурированный объект `RandomForestRegressor` главному оркестратору `main.py` для финального развертывания эксперимента на полной выборке `X_train`.

3 СРАВНИТЕЛЬНЫЙ АНАЛИЗ МОДЕЛЕЙ

Оценка эффективности алгоритмов выполнялась на основе трех ключевых критериев: средней абсолютной ошибки (MAE), коэффициента детерминации (R^2) и времени обучения/валидации модели в секундах.

Для обеспечения наглядности и сквозного сопоставления результатов все полученные метрики были сведены в единую аналитическую таблицу 3.1.

Таблица 3.1 – Сравнительные метрики эффективности алгоритмов

Архитектура модели	Время вычисления (сек)	Средняя абсолютная ошибка (MAE)	Коэффициент детерминации (R^2)
Linear Regression (Analytic)	0.0101	12.0895	0.5152
Linear Regression (SGD)	0.0646	12.0997	0.5141
Kernel Regression (RBF)	1.0259	8.9392	0.7017
Random Forest	2.1264	4.3512	0.9180

Для наглядного сопоставления результатов и выявления баланса между точностью аппроксимации и вычислительной эффективностью алгоритмов был сформирован трехпанельный гистограммный комплекс рисунок 3.1.

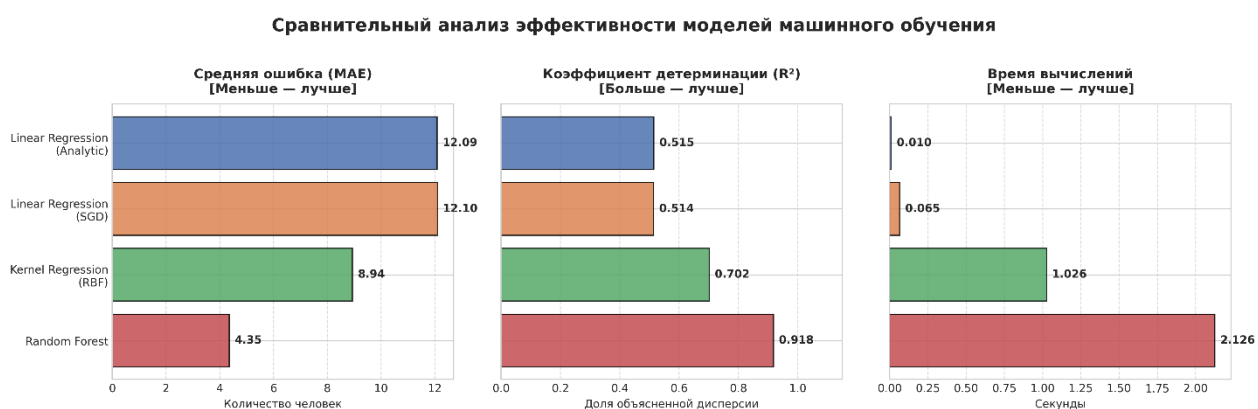


Рисунок 3.1 – Распределение метрик качества (MAE, R^2) и времени работы исследуемых моделей

Анализ метрик качества и вычислительной сложности

Сопоставление численных значений из Таблицы 3.1 позволяет выявить важные закономерности, обусловленные внутренней математической структурой каждого алгоритма.

Линейные модели (Analytic vs SGD).

Качество аппроксимации: Обе модели продемонстрировали практически идентичные и самые низкие результаты среди всех исследуемых алгоритмов (MAE 12.1, R^2 0.515). Коэффициент детерминации на уровне 51.5% свидетельствует о том, что линейная спецификация способна уловить лишь самый общий глобальный тренд (общий рост загруженности зала к вечеру), но полностью игнорирует локальные пики и спады посещаемости. Налицо классическое недообучение (underfitting) из-за высокого математического смещения.

Непараметрическая ядерная регрессия (Kernel Regression).

Переход к нелинейному сглаживанию через RBF-ядро и аппроксимацию Нистрёма позволил радикально повысить качество модели. Ошибка MAE снизилась с 12.1 до 8.94 (улучшение на 26%), а метрика R^2 выросла до 0.7017. Модель оказалась способна объяснить уже 70.2% дисперсии целевой переменной.

Ансамблирование (Random Forest).

Модель Случайного леса показала феноменальную точность, снизив среднюю ошибку MAE до 4.35 человек, что почти в 3 раза лучше показателей линейной регрессии. Коэффициент детерминации составил 0.9180, что по общепринятым шкалам оценки качества моделей классифицируется как отличный уровень аппроксимации.

Высокое качество обусловлено способностью композиции глубоких деревьев идеально изолировать сложные нелинейные паттерны в данных (взаимосвязи типа «день недели – час суток – праздничный день») без риска переобучения благодаря бэггингу.

Для проведения качественного визуального анализа работы алгоритмов был построен четырехпанельный график зависимости предсказаний моделей от времени суток. На графиках серыми маркерами отображены реальные исторические точки загруженности тренажерного зала, а красной линией – прогнозная траектория модели, рисунок 3.2.

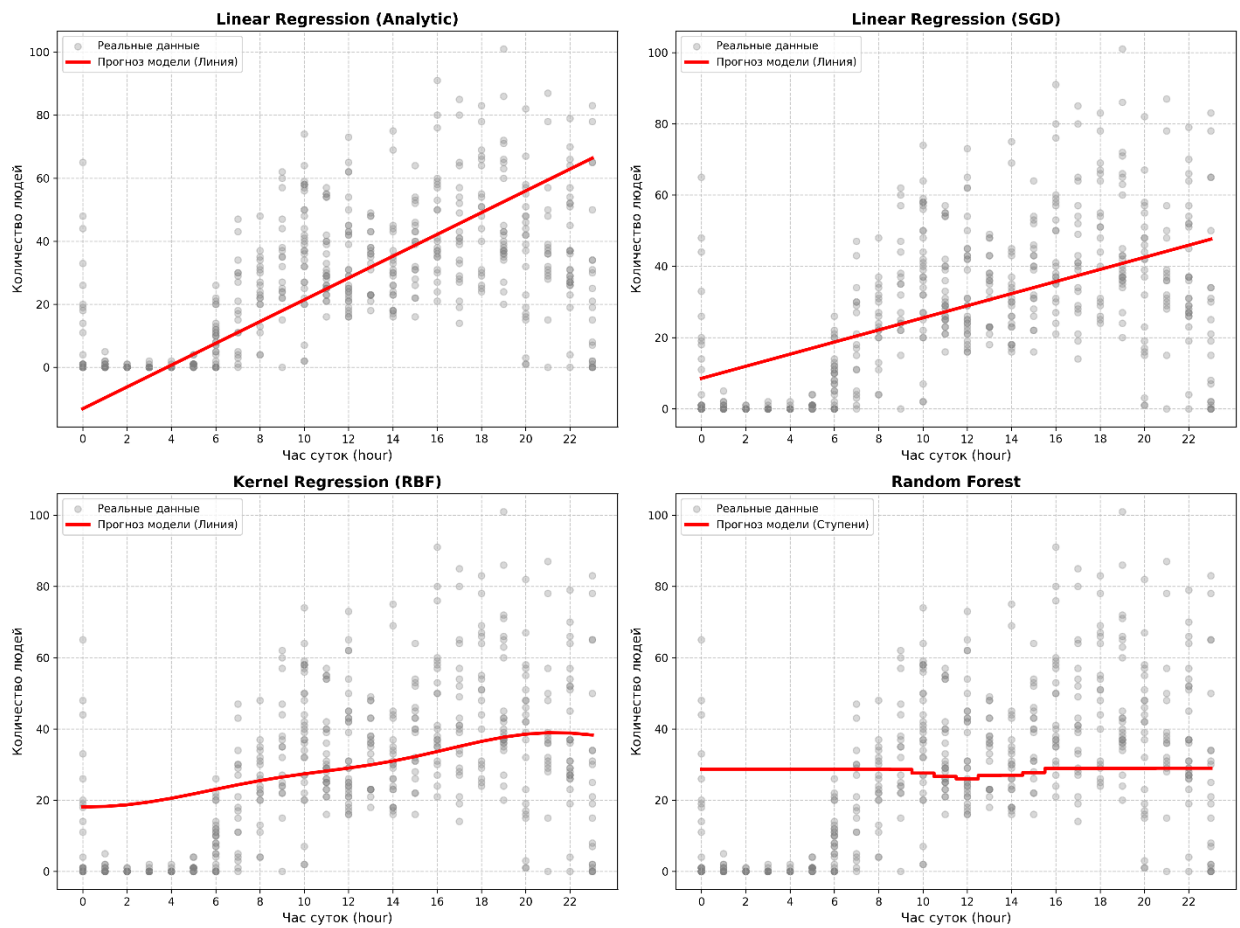


Рисунок 3.2 – График зависимости предсказаний моделей от времени суток

ЗАКЛЮЧЕНИЕ

Для задачи прогнозирования посещаемости спортивного объекта классические линейные подходы неприменимы из-за сложного нелинейного характера спроса. В качестве временного и легкого решения для плавного тренда подходит ядерная регрессия. Однако для реального промышленного внедрения в системы автоматизации фитнес-центров безоговорочно рекомендуется Случайный лес (Random Forest), обеспечивающий прецизионную точность при допустимых вычислительных затратах.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

[1] MachineLearning.ru [Электронный ресурс]. – Машинное обучение (курс лекций, К.В. Воронцов). – Режим доступа: http://www.machinelearning.ru/wiki/index.php?title=%D0%9C%D0%B0%D1%88%D0%B8%D0%BD%D0%BD%D0%BE%D0%B5_%D0%BE%D0%B1%D1%83%D1%87%D0%B5%D0%BD%D0%B8%D0%B5_%28%D0%BA%D1%83%D1%80%D1%81_%D0%BB%D0%B5%D0%BA%D1%86%D0%B8%D0%B9%2C%D0%9A.%D0%92.%D0%92%D0%BE%D1%80%D0%BE%D0%BD%D1%86%D0%BE%D0%B2%29

[2] GitHub [Электронный ресурс]. – Практические материалы и семинары по машинному обучению (MachineLearningSeminars). – Режим доступа: <https://github.com/andriygav/MachineLearningSeminars>